CS201 Spring 2026 — Lecture Notes

Clarissa Littler

Spring 2026

Preface

This document is a running narrative of what we covered in each class meeting this term. Each section pairs the agenda for the day with the little code samples we actually typed up together, so if you missed a lecture or want to go back and re-read, the whole arc of a day should be here in one place. The source files themselves live in the `lecture`*N*`/` subdirectories of the class repo, and all of the snippets quoted below are verbatim from those files.

Contents

1 Lecture 1 — Welcome, C vs. C++, Bitwise Tricks, and a Peek at Integers

This one was mostly an orientation plus a crash-course in the parts of C that tend to trip up people coming from C++. I was genuinely a little surprised everyone showed up, so thank you for that!

Logistics

The class is my own personal admixture of four things:

- C
- x86-64 assembly
- Concurrency
- Operating systems theory and practice

Grading-wise, you've got quizzes, discussions, and coding assignments. The class repo is going to be your friend: that's where the textbook lives, where all the "Agenda" files like this one come from, and where we'll drop every bit of code we work on in class.

Structurally, I'll aim to start about five minutes after 10, break after every 50 minutes, and try not to keep you much past two hours.

C: What's different?

If you're coming from C++, the big shocks are:

- No pass-by-reference. You get pass-by-value or you get pointers, full stop.
- No `bool` type (well, not in the way you're used to — nonzero is truthy and zero is falsy, and there's no native `true`/`false`).
- No `nullptr`; you use the `NULL` macro (which is basically just `0` cast to a pointer).
- I/O is `printf`/`scanf` rather than streams.

We played with two small files to make this concrete. The first, `arraything.c`, shows that arrays do decay to pointers — so you pass the length by hand — and that you iterate with a plain `for` loop rather than an iterator:

```
void arrayMunger(int a[], int s){
  for(int i=0; i<s; i++){
    a[i] = 2*a[i];
  }
}
```

The little commentary `// arr[i] ===> *(arr + i)` hints at the thing we'll lean on all term: indexing is just pointer arithmetic in disguise.

The second file, `passPointy.c`, makes the "no pass by reference" point tangible. If you want a function to modify its caller's variable, you pass the *address* and dereference inside:

```
void intMunger(int* p){
  *p = 2*(*p);
}
```

And you call it with `intMunger(&test);`. That ampersand is doing real work — it's giving you a pointer to `test` rather than a copy of it.

Bitwise operations

We went through the usual suspects with truth tables:

- `&` (AND), `|` (OR), `^` (XOR), `~` (NOT).
- `<<` (shift left, pads with zeroes, anything that falls off the edge is gone) and `>>` (shift right, same idea on the other side).

The file `bitmunger1.c` is where this all came together. It defines a little loop that prints out the 32 bits of an `int`, lets you pick a bit to toggle, and optionally negates the integer via two's complement:

```c
void toggleBit(int* num,int place){
  *num = (*num) ^ (1 << place);
}

void twosComplement(int* num){
    *num = ~((*num) - 1);
}
```

`toggleBit` is the canonical XOR trick: XOR-ing a bit with 1 flips it, XOR-ing with 0 leaves it alone. `twosComplement` implements the "subtract one, then invert" rule for negating a two's-complement integer.

Peeking inside an integer: two's complement

We derived why two's complement works. A plain unsigned n-bit integer encodes

$$\sum_{i=0}^{n-1} b_i \cdot 2^i,$$

so `11111111` (8 bits) is $128 + 64 + \cdots + 1 = 255$. In two's complement, the top bit's weight flips sign:

$$-b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i.$$

That gives you one clean mental check: all-ones is always -1, because $-2^{n-1} + (2^{n-1} - 1) = -1$. And to negate a number, you subtract one then invert all the bits (or equivalently, invert and then add one).

A preview of floats

We ran out of time before we could fully explore floats, but I showed off `floatmunger1.c`, which is structurally the same as `bitmunger1.c` but reinterprets an `int`'s bits as a `float` via a pointer cast. The little spaces it prints (between bits 31/30 and 23/22) are hinting at the IEEE-754 sign / exponent / fraction split we'll dig into next time.

2 Lecture 2 — Hex, Endianness, IEEE-754, and Where Floats Lie to You

The plan was to finish the float discussion we previewed on day one, but before diving in we took a small detour through hex and endianness.

Warm-up: hex codes and the shape of a pointer

Hex is base-16, digits 0–F. The reason it shows up everywhere — addresses, colors, byte dumps — is that two hex digits are exactly one byte ($16 \cdot 16 = 256 = 2^8$). So when you see an RGBA color like `#8765AF`, that's really three (or four) bytes: one for red, one for green, one for blue, optionally one for transparency.

Endianness

We wrote `byteOrderTest.c` to actually look at how an 8-byte integer is laid out in memory on our machines:

```
long int num = 0x0123456789abcdef;
unsigned char* p = (unsigned char*)&num;

printf("The bytes are stored in memory as: ");
for(int i=0; i<8; i++){
  printf("%x ",*(p+i));
}
```

On an x86-64 box this prints the bytes out in the opposite order from how we wrote the literal — classic little-endian. The trick is the `(unsigned char*)` cast: once we have a byte-pointer, pointer arithmetic moves us one byte at a time, so we can walk the actual storage.

Sizes and pointers

Alongside that, `sizes.c` just prints `sizeof(int)`, `sizeof(long)`, `sizeof(char)`, `sizeof(float)`, `sizeof(double)`, and the size of a pointer. On our 64-bit Linux boxes: int is 4, long is 8, char is 1, float is 4, double is 8, and any pointer is 8. That last one is the important bit — *all* pointers are the same size, regardless of what they point to, because they're all just addresses.

And `pointerArith.c` drove home that pointer arithmetic scales with the *pointee* size. An `int*` advances 4 bytes per `+1`, a `char*` advances 1, a `char**` advances 8, and so on.

IEEE-754, formally

We did the 32-bit (`float`) version because the 64-bit version is just "same thing with bigger fields."

- 1 bit sign
- 8 bits exponent (stored with a bias of 127)
- 23 bits fraction

In the normal case, the value is

$$(-1)^s \cdot 2^{e-127} \cdot (1+f),$$

where f is the fractional part read as "rational binary": the bit at position k after the implicit point contributes 2^{-k}.

Worked example: `0 10000001 1010...0` is

$$(-1)^0 \cdot 2^{129-127} \cdot (1+\tfrac{1}{2}+\tfrac{1}{8}) = 4 \cdot \tfrac{13}{8} = 6.5.$$

We exercised this live with `floatmunger1.c` (same idea as lecture 1, but now with `%.16f` so you can see lots of digits). Flipping specific bits and watching the printed value change is the single best way to internalize the format.

Edge cases

- Exponent is all 1s:
  - fraction is zero ⇒ $\pm\infty$,
  - fraction is nonzero ⇒ `NaN`.
- Exponent is all 0s (subnormal numbers): the exponent sticks at 2^{-126}, but the leading implicit 1 is gone, so the value is $(-1)^s \cdot 2^{-126} \cdot f$.

Where floats lie to you

`floatlimit.c` is the "gotcha" example:

```
float f = 0.0001;
float accum = 0;
for(int i=0; i<10000; i++){
  accum += f;
}
printf("%.20f\n",accum);
```

We'd love this to print `1.00000000000000000000`. It doesn't. The reason is a mix of: 0.0001 is not exactly representable in binary, and we're accumulating that small rounding error 10,000 times.

Why is this *unavoidable*? A bit of math: the rationals and the integers are countably infinite, but the reals form a strictly larger uncountable set. There is no way to cover the reals with finitely many bits, or even with countably many bits; we're forced to pick a countable scatter of "breadcrumbs" and round every other real to the nearest one.

Round to even

To keep that rounding error from biasing one direction, IEEE-754 specifies "round to even": when a real is exactly halfway between two breadcrumbs, the hardware rounds to the one whose last bit is zero. Over many operations this rounds up about as often as it rounds down.

3 Lecture 3 — `malloc`, and Your First Real Assembly

The real headline today: we left C-land and wrote actual assembly against the bare Linux syscall interface. But first, I owed you `malloc`.

`malloc`, finally

In my rush last time I completely skipped dynamic allocation, which is embarrassing, so we opened with two quick examples.

`malloc.c` is the minimum viable pointer dance:

```
int* n = malloc(sizeof(int));
*n = 10;
printf("%d\n",*n);
free(n);
n = 0;
if(0 == NULL){
```

```
  printf("Yup\n");
}
```

Two things to note. First, `malloc` always returns a pointer and always needs you to tell it how many bytes you want — unlike `new T` in C++, it doesn't know anything about your type, so you pass `sizeof(int)` (or a multiple of it). Second, that final `if` is my excuse to point out that `NULL` is literally just `0` cast to a pointer type.

`malloc2.c` extends this to an array and a struct:

```
int* narr = malloc(100*sizeof(int));
narr[2] = 5;

struct Goofus* g = malloc(sizeof(struct Goofus));
g->g1 = 5; // same thing as (*g).g1
```

Once you've got a pointer from `malloc`, you can index it just like an array. The `->` arrow is shorthand for "dereference then access a field."

Stepping back: what is a CPU, anyway?

The big conceptual pivot for the rest of the term:

- A CPU is, at bottom, a thing that fetches *instructions* from memory and does what they say.
- The set of instructions it understands is an *instruction set architecture* (ISA). Ours is x86-64.
- Assembly language is a one-to-one (ish) textual encoding of those instructions, so writing assembly is as close to "talking to the CPU" as we'll practically get.
- Registers are a tiny amount of extremely fast storage inside the CPU itself. Most instructions operate on registers; memory comes in via load/store.
- Memory hierarchy, the 30-second version: registers → L1/L2/L3 cache → main memory → disk. Each level is roughly an order of magnitude slower and roughly an order of magnitude bigger than the one above it.

I also mentioned `gcc -S`, which tells gcc to stop at assembly rather than assemble and link. We'll come back to it.

Your first bare assembly program

`first.s` is deliberately broken:

```
    .section .text
    .global _start

_start:
    mov $60,%rax
```

This is the shortest thing that looks like an assembly program. The `.text` section is where code lives; `_start` is the symbol the linker uses as the program entry point (unlike in a C program, there is no runtime to call `main` for us).

We moved `60` into `%rax` — 60 is the syscall number for `exit` on x86-64 Linux — and then... did nothing else. Because we never actually issued the `syscall` instruction and there's no implicit return, the CPU just wandered off the end of our code and trapped. Hence the crash.

`firstfixed.s` is the corrected version:

```
_start:
    mov $60,%rax
    mov $-1,%rdi
    syscall
```

Syscall convention: put the syscall number in `%rax`, put arguments in `%rdi`, `%rsi`, `%rdx`, etc., and then execute `syscall`. For `exit(status)`, the status goes in `%rdi`. So this exits with status `-1`, and `echo $?` in the shell will confirm it.

`add` and `sub`

`firstadd.s` and `firstsub.s` are our first binary operations:

```
_start:
    mov $10,%rbx
    mov $20,%rcx
    add %rbx,%rcx
    mov %rcx,%rdi
    mov $60,%rax
    syscall
```

Syntax note that will bite you all term: in AT&T syntax, `add S, D` means `D = D + S`. The destination is on the *right*. So `add %rbx,%rcx` leaves 30 in `%rcx`. `sub` is analogous: `sub %rbx,%rcx` computes `D - S` and leaves 10 in `%rcx`.

We also did the fun thing of running `gcc -S test.c` on a tiny C file and skimming the output (`test.s`). It's noisy (`.cfi_`directives, frame setup, PLT calls) but the actual work is recognizable: move 10 into a local, load it into `%esi`, load the format string's address via `leaq`, call `printf@PLT`.

4 Lecture 4 — Registers, Calling Convention, Addressing Modes, Control Flow, and "Hello, world"

Today was a dense one: we filled in the register table, introduced the Linux x86-64 calling convention, walked through every addressing mode we'll use for arrays, covered `cmp`/`jmp`, wrote a for-loop in assembly, and finished by doing `write` and `exit` syscalls by hand.

Logistics

The Linux server admin is in the process of activating accounts for folks who can't log in yet, so hang tight if that's you.

Register reference

The basic x86-64 general-purpose registers and what they were historically used for:

- `%rax` — accumulator (return values, implicit operand for `mul`/`div`).

- `%rbx` — base register (historically an array base).
- `%rcx` — counter (loops, `rep`).
- `%rdx` — data (I/O, upper half of multiplication/division).
- `%rsi` — source index (string ops).
- `%rdi` — destination index (string ops).
- `%rbp` — base pointer (stack frame base).
- `%rsp` — stack pointer (top of the stack).
- `%r8`–`%r15` — extra general-purpose registers.
- `%rip` — instruction pointer; we don't write to this directly, but we *do* use it in RIP-relative addressing.

The floating-point/SIMD registers (`%xmm0...`) live in the `assemblyGuide.org` in the repo.

Linux/x86-64 calling convention, first pass

- Return values go in `%rax`.
- The first six integer/pointer arguments are passed in, in order: `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`.

Syscalls use a slightly different convention (the fourth argument is `%r10`, not `%rcx`), but for function calls in user code the list above is what you want.

`add`/`sub`, again

We re-ran `firstadd.s` from last time as a warm-up; the only new thing is a pair of clarifying comments explaining that `%rdi` is the exit-code argument and `60` in `%rax` selects the `exit` syscall.

Global variables: values, pointers, and `lea`

Three files building on each other:

`addGlobal.s` — define a global with `.quad`, then load, add, store, load-again:

```
    .section .data
num: .quad 200

_start:
    mov num,%rbx # load num into %rbx
    add $10,%rbx
    mov %rbx,num # store back
    mov num,%rdi
    mov $60,%rax
    syscall
```

This works, but referencing a symbol like `num` directly makes the assembler emit an absolute address, which is awkward for position-independent code.

`addGlobalBetter.s` — same thing, but using *RIP-relative* addressing:

```
    mov num(%rip),%rbx
    add $10,%rbx
    mov %rbx,num(%rip)
```

Here `num(%rip)` means "the address that is `num`-minus-the-next-instruction bytes away from the current instruction pointer." The assembler computes that offset for us at build time. This is the form you want in modern code.

`addGlobalLea.s` — now using `lea` to take *the address of* `num`, the way `&` does in C:

```
    lea num(%rip),%rbx # rbx = &num
    addq $10,(%rbx) # *rbx = *rbx + 10
    mov num(%rip),%rdi
    mov $60,%rax
    syscall
```

The comment block at the bottom of the file spells out the C equivalent:

```
int num = 200;
int* nump = &num;
*nump = *nump + 10;
```

Parentheses around a register mean "dereference." So `(%rbx)` is "the 8 bytes of memory starting at the address held in `%rbx`."

From a scalar to an array

Four files, one new addressing idea at a time.

`addArray1.s` — change `num` to an array of four quadwords. The code is unchanged, so it just touches the first element.

`addArray2.s` — to get at element 1, add 8 to the pointer (because one `.quad` is 8 bytes):

```
    lea num(%rip),%rbx
    addq $8,%rbx
    addq $10,(%rbx)
    movq (%rbx),%rdi
```

`addArray3.s` — same effect, but using the full "scale-index" addressing mode:

```
    lea num(%rip),%rbx
    mov $1,%rcx
    addq $10,(%rbx,%rcx,8) # *(base + index*8)
    movq (%rbx,%rcx,8),%rdi
```

The syntax `(base, index, scale)` computes `base + index*scale`. The scale has to be 1, 2, 4, or 8. This is exactly how C-style array indexing compiles — you'll see the same form over and over.

`addArray4.s` — same as `addArray2`, but with `.long` (4-byte) elements instead of `.quad`, which forces us to use the `l` suffix (`addl`, `movl`) and the 32-bit register name `%edi`.

Note the subtle point: with `.long`, a step of "one element" is 4 bytes, not 8. In the file as written, we advanced the pointer by 8 bytes, which lands us at element *two* of the `.long` array, not element one. That kind of size/stride mismatch is a classic bug I'll keep harping on all term.

`cmp` and `jmp`

`cmp S, D` does (roughly) `D - S` and sets flags without saving the result anywhere. The conditional jumps then read those flags:

- `jmp` — unconditional.
- `jl` — $D < S$. (Yes, `cmp S, D` then "if less" compares D to S; it still surprises me.)
- `jle` — $D \leq S$.
- `je`, `jne` — equal, not equal.
- `jg`, `jge` — greater, greater-or-equal.

The demo, `cmp1.s`:

```
    mov $10,%rbx
    mov $20,%rcx
    mov $-1,%rdi
    cmp %rbx,%rcx # flags set as if computing %rcx - %rbx
    jge greater
    mov $1,%rdi
greater:
    mov $60,%rax
    syscall
```

Since $20 \geq 10$, we jump to `greater` and exit with status `-1` (or `255` as far as the shell is concerned).

A for-loop in assembly

Two flavors of "sum the numbers from 1 to 10."

`sumLoop.s` counts up:

```
    mov $0,%rbx # accumulator
    mov $1,%rcx # counter
loopStart:
    add %rcx,%rbx
    add $1,%rcx
    cmp $10,%rcx
    jle loopStart
```

`sumLoopB.s` counts down:

```
    mov $0,%rbx
    mov $10,%rcx
loopStart:
    add %rcx,%rbx
    sub $1,%rcx
    cmp $0,%rcx
    jg loopStart
```

Both leave 55 in `%rbx`, which we then move into `%rdi` so we can read it out as the exit status.

Buffers, strings, and `write`

The grand finale, `hello.s`:

```
    .section .data
msg: .asciz "Hello world!\n"
hellolen = . - msg

    .section .text
    .globl _start

_start:
    mov $1,%rax # syscall number for write
    mov $1,%rdi # fd = 1 (stdout)
    lea msg(%rip),%rsi # buffer address
    mov $hellolen,%rdx # number of bytes
    syscall

    xor %rdi,%rdi # exit status 0
    mov $60,%rax # syscall number for exit
    syscall
```

Points of interest:

- `.asciz` lays down the characters followed by a null terminator.
- `hellolen = . - msg` is a classic assembly idiom: the dot is "current address," so subtracting the start of `msg` gives the length in bytes, including the newline and the terminator. We use that as the third argument to `write`.
- `write` is syscall 1 on Linux x86-64. Arguments are `(fd, buf, count)` in `%rdi`, `%rsi`, `%rdx`.
- `xor %rdi,%rdi` is the idiomatic way to zero a register — one byte shorter than `mov $0,%rdi`.

Next time we'll close this out with an `echo`-style program that *reads* from stdin and writes back, which means handling buffers properly.